Agentic Development Workflow

Breaking down complex features into logical layers

KavinSchool

Decomposing Layers (1/3): Research

Type: Sequential

Goal: Map the codebase and identify bounded contexts.

- The first agent is strictly read-only.
- It identifies schemas, hubs, and dependencies.
- It does not write any code.

> **Why?** Writing code before mapping dependencies leads to conflicting architectures.

Example: Research Layer

> **CLAUDE-HELPER EXAMPLE**

The `context-collector` Agent

When initiated, this agent restricts itself to reading the workspace.

- **Action:** Runs commands like `git grep` and parses package configurations.
- **Output:** Generates a structured `context-brief.md` file.
- **Why it matters:** The output serves as the single source of truth for the planner, preventing hallucinatory architecture.

Decomposing Layers (2/3): Builders

Type: Parallelizable

Goal: Implement isolated feature domains.

- **Frontend Builder:** Works strictly on UI/client code.
- **Backend Builder:** Works strictly on APIs and services.
- Both builders operate concurrently.

> **Condition:** They must share an agreed-upon API contract and data schema beforehand.

Example: Builders Layer

> **CLAUDE-HELPER EXAMPLE**

The `implementor` and `test-writer` Agents

Once the contract is approved, the orchestrator spawns multiple agents at once.

- **Isolation:** The `implementor` only receives access to the source directory (`src/`).
- **Parallelism:** The `test-writer` is spawned simultaneously but only has access to the test directory (`tests/`).
- **Why it matters:** They work concurrently in separate Git worktrees without colliding.

Decomposing Layers (3/3): Validation

Type: Parallelizable

Goal: Verify the code across multiple dimensions.

- **API Tests:** Ensure the backend endpoints work.
- **UI Tests:** Ensure the frontend components render.
- **Performance Tests:** Run `k6` to validate load times.
- These run independently but report to a single handoff summary.

Example: Validation Layer

> **CLAUDE-HELPER EXAMPLE**

The `quality-gate-runner` Agent

Runs real quality gates on the completed builders' code sequentially.

- **Step 1:** Runs the build and unit tests.
- **Step 2:** Runs Playwright UI and API tests.
- **Step 3:** Runs `k6` load testing.
- **Why it matters:** The orchestrator halts the process if any gate fails, routing the failure back to the specific worker who broke it.

Find Safe Boundaries

Parallelism starts with low coupling, not agent count

KavinSchool

A Feature Is Not a Task

A feature brief states a product outcome across several concerns.

An agent-ready task produces one named, observable deliverable.

Boundary test: Can another agent act without hidden context?

Evidence test: Can another person verify completion from artifacts?

Inspect Before You Split

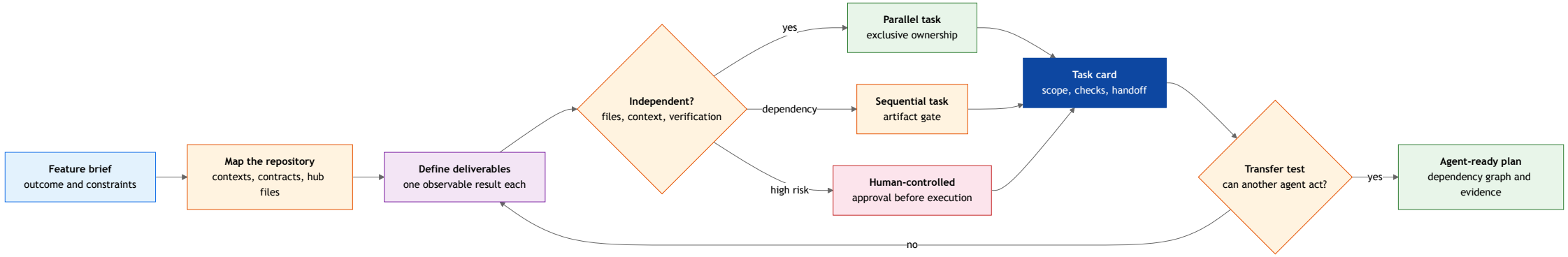
1. Map bounded contexts and existing contracts.
2. Find schemas, routers, manifests, and migrations.
3. Name the available verification commands.
4. Identify decisions with high blast radius.
5. Split by cohesive deliverable, not file count.

Interactive: Repository Architecture

A real example of what "mapping bounded contexts" looks like: the source modules of a repository (rules, commands, agents, skills, hooks, plugins, docs) and how one module travels through the installer into a live session.

Open `repo-architecture-overview.html` from `claude-helper/docs/interactive/` to trace the source-modules view and the install-path view.

From Brief to Agent-Ready Plan



What Does the Task Card Do?

A task card carries enough context to assign and verify work without replaying the planning conversation.

CONTROLS WORK

Scope

Objective, deliverable, allowed files, and non-goals.

ORDERS WORK

Coordination

Dependencies and prerequisite decisions.

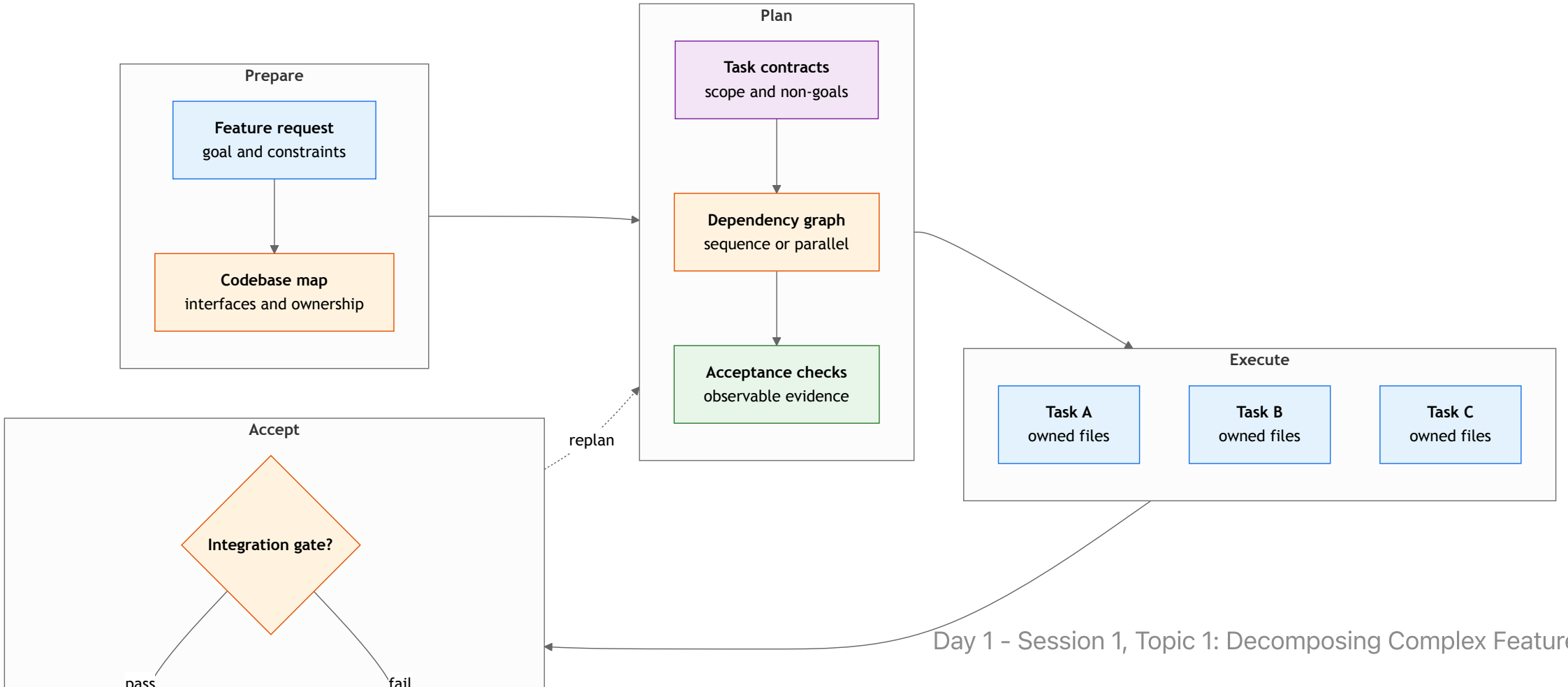
VERIFIES WORK

Proof

Acceptance check and handoff evidence.

Generic Feature Decomposition Flow

REFERENCE ONLY



Dependency-Aware Task Decomposition (Financial Services)

REAL-WORLD SCENARIO

Financial services (AWS - AgentCore portfolio advisory reference):

- A portfolio query is divided among valuation, risk, market research, and advisor-synthesis agents.
- Each specialist owns one domain while an orchestrator combines results with authentication, tracing, and sandboxed code.

Enterprise Architecture Signals

Signal	Planning implication
Bounded context	Keep domain knowledge and ownership together.
Event or API contract	Approve the interface before consumers begin.
Gateway or service mesh	Keep platform operations outside feature tasks.
Security and CI gates	Automate security and independent checks before merge.
Hidden state	Expose dependencies before delegation.

Structural Hubs Change the Plan

Schemas, routers, manifests, and migrations connect otherwise separate work.

Example Hub (`router.py`):

```
def notify(event):
    validate_event(event)
    # ⚠️ CONFLICT RISK: Multiple agents adding their channels here
    renderers = {"email": render_email, "sms": render_sms}
    return {ch: renderers[ch](event) for ch in CHANNEL_ORDER}
```

Do not: Give every channel agent permission to edit `router.py`.

Do: Lock the contract, assign channel files, and reserve integration ownership.

> Shared-file conflicts often signal a missing dependency or ownership decision.

What is a Dependency Graph?

A **Dependency Graph** is a network that maps the required execution order of tasks to ensure safe, coordinated development. It visualizes which work is blocked, which can run in parallel, and where tasks integrate.

Why do we need it in Agentic Development?

- **Identify Blockers:** Exposes sequential bottlenecks (e.g., waiting for an API schema approval) before agents start working.
- **Find Parallelism:** Highlights independent tasks that are safe for concurrent agent assignment.
- **Prevent Conflicts:** Ensures hub files (`router.py`) are edited only after prerequisites finish.

> Generically, tasks flow from **Contracts (Upstream)** → **Independent Work (Parallel)** → **Integration (Downstream)**.

Write Transferable Task Cards

Replace conversational assumptions with durable contracts

KavinSchool

Task Card Contract

Field	Question it answers
Objective and deliverable	What observable result must exist?
Context and files	What may the agent inspect and change?
Dependencies	Which artifact or decision unlocks this task?
Acceptance	Which command proves the required behavior?
Non-goals	What must remain untouched?
Handoff	Which evidence transfers to the next owner?

Steps Are Not Acceptance Criteria

Weak: Add an email renderer and update its tests.

Strong: The supplied shipment fixture produces the expected subject and body when `python3 -m unittest tests.test_email` runs.

The first statement describes activity. The second defines observable evidence.

Classify the Work

Classification	Use when
Parallel	Files are exclusive, contracts are stable, and checks are independent.
Sequential	A consumer needs an upstream artifact or decision.
Human-controlled	Work is ambiguous, irreversible, sensitive, or high risk.
Not ready	The outcome, context, ownership, or verification path is missing.

Transfer Test

Give the task card to someone who missed the planning conversation.

Can they identify the deliverable, allowed files, dependencies, and command?

Can they report changed files, decisions, results, and unresolved risks?

If not, refine the card before assigning an agent.

Demo: Task Boundary Check

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-task-boundary-check.py`.

- Conflict count: naive tasks share the central router.
- Execution waves: the bounded plan exposes safe concurrency.
- Integration ownership: one task controls the hub and full check.

> Runs locally with the Python standard library; no API key needed.

Demo: Task Card Transfer Test

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-task-card-transfer.py`.

- Missing fields: conversational assumptions become visible.
- Acceptance quality: vague success differs from an executable command.
- Transferability: a durable card supports independent execution.

> Runs locally with the Python standard library; no API key needed.

Deriving Boundaries from Code Architecture

DEPENDENCY MAP

Import Graph

Shows how repository files depend on one another.

CONFLICT RISK

High Fan-Out

Files imported widely, such as `router.py`, need controlled ownership.

PARALLEL CANDIDATE

Leaf Node

Files with no internal imports are safer to assign independently.

> Generate the graph to find boundaries from code evidence, not memory.

Demo: Boundaries From the Real Import Graph

From the companion repo, run `python3 day1/demos/demo-boundary-from-imports.py`.

- Parsed graph: `ast` reads every module; `router.py` is found, not assumed.
- Hub detection: highest internal fan-out identifies the contested file.
- Waves: dependencies computed from the measured graph, then verified.

> Real parsing of the course codebase; no API key, runs in a temporary copy.
> Notebook: `day1/demos-notebook/demo-boundary-from-imports.ipynb`.

What the Demo Computes

The script derives file owners instead of printing a prepared judgment.

It rejects a cycle when no dependency wave can make progress.

The contract-first plan creates three waves: schema, channels, integration.

The result is evidence about the plan, not a promise that agents will coordinate.

Current Industry Evidence

Anthropic reports that multi-agent systems fit breadth-first, independent work best.

Its production guidance requires objectives, output formats, and clear boundaries.

A 2026 study reported gains from cohesion-aware partitioning over tested baselines.

> Reported benchmark gains are research results, not guarantees for every repository.

Context Assembly

> **TRICK OF THE TRADE**

Instead of manually listing files for an agent to read, orchestrators use real-world tooling to build context automatically:

- **Dynamic Discovery**: Use `git grep -l "class Router"` or `tree src/` to find relevant files.
- **AST Parsing**: Use simple Python/Bash scripts to extract just the function signatures or schemas, not the entire codebase.

Context Assembly

- **Automated Injection**: Pipe this context into the agent's CLI:

```
claude -p "Implement the feature based on this context: $(cat context.txt)"
```

> **Pro Tip:** Never assume a human will manually read the task card and type out the file paths. Script it

Lab 1.1: Agent-Ready Work Plan

Open `day1/lab1_1_agent-ready-work-plan/` in the companion repo.

Goal: Produce a machine-checked plan for retail shipment notifications.

1. Lock the event contract and map the hub files.
2. Write bounded task cards with supplied acceptance commands.
3. Classify dependencies and record a human approval point.

> This is a planning lab. Do not implement the notification system.

Key Takeaways

1. Agent-ready work has one observable deliverable and a verification path.
2. Stable contracts and exclusive ownership make parallel execution safer.
3. Dependency edges must name artifacts or decisions, not vague sequencing.
4. Human control belongs at ambiguous, sensitive, and high-risk boundaries.
5. A durable handoff carries evidence instead of full chat history.

Ready for Lab 1.1?

Build the plan before assigning the agents.

KavinSchool